

模拟器验证与开源对照报告

64 项严格检查、3 个真实缺陷、1 个把两轮调参全部作废的陷阱
——以及同类开源实现的横向调研

2026 高教社杯全国大学生数学建模竞赛 B 题

2026 年 9 月 11 日

摘要

本文档回答三个问题：我的本地模拟器到底可不可信、有没有同类开源实现可以对照、这一轮“优化”里哪些结论是真的。

我们写了一个 64 项检查的验证套件（code/verify_simulator.py），覆盖 HTTP 协议、虚拟时间记账、物理判据、响应契约、与开源参考的成本模型交叉对照、守恒律、增量认证的等价性。结果是 64/64 通过，但过程中抓出并修复了 3 个真实缺陷：

1. robot_id 在服务端未指定时形同虚设（HTTP 层只检查启动参数，不检查首次 /enter 绑定的身份）；
2. 示向度四舍五入到两位小数后可以突破 $\pm 1^\circ$ 的硬上界（最大超出 0.0024° ）；
3. certification_scan 的“未认证位置”列表被一个早退优化截断，规划器长期在残缺信息上决策。

我们还找到了一份专门针对本题的开源工程（官方模拟器的驱动 + 离线 mock），逐项对照了 12 个物理/时间常量与完整的成本模型：逐步完全一致；唯一分歧在示向度误差模型——该 mock 每次测量重新随机，违反附件 1“同一地点重复测量结果相同”的题设，我们的实现按题设做成了地点函数。

最后如实记录一个把两轮参数寻优全部作废的陷阱：算例池只生成一次并复用，而 Arena 会就地清除 Source 对象，导致第二次以后的每次评估都跑在“已经被清空的算例”上，于是出现了一个并不存在的“44% 提升”。加入可复现性守卫后重跑，真实收益是 4%（问题三）与 20%（问题四）。

目录

1 为什么必须验证模拟器	3
2 64 项检查的构成	4
3 抓到的三个真实缺陷	5
3.1 缺陷 1: robot_id 绑定形同虚设	5
3.2 缺陷 2: 四舍五入突破 $\pm 1^\circ$ 硬上界	5
3.3 缺陷 3: 认证扫描的“未认证位置”列表被截断（最有价值的一条）	5

4	与开源实现的交叉对照	6
4.1	调研结果	6
4.2	逐项常量对照：12/12 相等	7
4.3	成本模型：300 步随机动作逐步相等	7
4.4	唯一分歧：示向度误差模型	7
5	一个把两轮调参全部作废的陷阱	8
5.1	现象	8
5.2	根因与最小复现	8
5.3	修法与守卫	9
5.4	修正后的真实收益	9
6	本轮算法改动清单	10
7	如何复现全部验证	10

1 为什么必须验证模拟器

本题的全部结论——完成率、平均定位清除时间、与基线策略的对比——都建立在一个前提上：**本地模拟器与官方模拟器在协议和计时上等价**。如果这个前提不成立，那么论文里所有的数字都只是在描述我自己的程序。

官方模拟器无法获取（百度网盘分发 + 联网注册 + 正式测试额度绑定账号），因此”验证”必须换一种做法：**不验证”是否与官方二进制一致”（做不到），而是验证”是否严格遵守附件 1 与附件 2 的每一条文字规定”**，并把每一条规定写成一条可执行、可复现、会失败的检查。

验证的设计原则

(1) **检查必须能失败**：每条检查都断言一个具体数值或状态，而不是”没报错就算过”。

(2) **区分”实现错”与”测试错”**：第一次运行 60 项里失败 10 项，其中 6 项事后证明是我的测试写错了（把绝对坐标当位移、在已开启的会话上测”未进场”语义等），只有 4 项是实现缺陷。两类错误必须分开处理，否则会为了”让测试变绿”而改坏实现。

(3) **优化必须自带等价性证明**：任何为了速度而重写的关键逻辑（本文是增量认证扫描），都要有一条检查与”从零重算”逐次比对。

2 64 项检查的构成

组	覆盖范围	项数	代表检查
A	HTTP 协议	18	四条路由、Content-Type/Encoding 校验、请求体上限、JSON 合法性、字段白名单、arena_id/robot_id 绑定、坐标与频道取值域、幂等重放、未进场/重复进场/退出后调用
B	虚拟时间记账	8	移动 = $d/5$ 、切换 1 s (同频道 0 s)、检测固定 5 s、清除命中 5 s/未命中 3 s、清除不改频道、时钟单调、已清除源静默
C	物理判据	16	源个数 10 ~ 16、频道互异、 $R_{\text{eff}} \in [1000, 1500]$ 、面积均匀采样、超距静默、5 m 内 near、误差上界、同点重测一致、清除半径 20 m 边界、定向半平面含边界、反侧静默、清除与角度无关
D	响应与日志契约	8	/enter 回传三个时限、measure_result 三态、clear_result 两态、日志五个必备字段与成本分解
E	开源参考交叉对照	5	12 个常量逐项相等、300 步随机动作逐步成本相等、算例统计量一致、误差模型分歧记录
F	守恒律	4	日志分量之和 = 最终虚拟时钟、行程守恒、单局通关、策略时钟不低于竞技场时钟
G	增量认证等价性	5	每次扫描与暴力重算逐次比对 (60 次)、插桩不改变单局结果、边界外补测点确实在域外、补测点数受限、内部候选不触发域外

运行方式与产物：

```

1 cd code
2 python verify_simulator.py      # 64 项检查，全过返回 0，否则返回 1
3 # 报告写入 data/simulator_verification.json

```

3 抓到的三个真实缺陷

3.1 缺陷 1: robot_id 绑定形同虚设

检查	robot_id_mismatch:用错误的 robot_id 发 /measure,应当返回 200 + accepted=false
实测	200 + accepted=true (被接受了)
根因	HTTP 层只检查服务启动时传入的 robot_id。默认启动(不传)时该值为 None,判断被整体跳过
修法	竞技场在首次成功 /enter 时绑定该 robot_id (官方运行时本来就”认识参赛队”),HTTP 层同时查询启动参数与绑定值
影响	不影响本队成绩(我们从不发错 id),但会让模拟器作为”公共 mock”时语义失真

3.2 缺陷 2: 四舍五入突破 ±1° 硬上界

检查	bearing_error_bound: 400 个有效样本的真实误差绝对值必须 ≤ 1°
实测	最大 1.0024°
根因	误差在 ±1° 内均匀采样后再四舍五入到两位小数,边界样本被舍入推到区间外
修法	新增 _round_within_error(): 舍入后若偏离真值超过上界,就按 0.01° 步长往回收,直到落回界内
影响	官方 mock 与我们的实现都有两位小数格式;修正后 20000 个样本最大误差 0.99985°。这条修正的意义在于: ±1° 是题面的硬约束,不应因为显示格式而被突破

3.3 缺陷 3: 认证扫描的”未认证位置”列表被截断(最有价值的一条)

认证扫描原本的实现里有一句看似合理的早退:

```
1 for q in grid:
2     for c in channels:
3         if flags.get(c) is False: # 该频道已经确定未通过
4             continue             # <-- 跳过它,但 q 也不再被记入 uncov
5         ...
6         if not ruled_out:
7             flags[c] = False
8             pending.append(c)
```

代码 1 原实现中的早退(已移除)

当一个频道在某个候选位置失败后,它在后续所有候选位置都被跳过,于是这些位置不会进入 uncov。这意味着:

- ok = (len(uncov) == 0) 仍然正确(只要有一个点失败就非空),所以终止判据没错;
- 但规划器看到的”未认证区域”只是真实区域的很小一部分——实测一次调用里真实未认证点 1253 个,而返回的只有 1 个。

重写为单调增量版本后（”已认证”只可能从否变是，因此每个候选点一旦被排除就永不回头；每来一条新读数，只需对仍然开放的候选点重新判定），既修好了完整性，又把单局 CPU 从 41 s 降到 2.8 s（15 倍）。

这条缺陷是怎么被发现的

它不是我读代码读出来的，而是第 G 组检查逼出来的：该组要求”增量结果必须等于从零重算的结果”。第一次运行立刻报 call 1: brute 1 vs incremental 1253 uncovered。追下去才发现错的不是新的增量实现，而是我用来当基准的旧逻辑——这也说明”写一条与实现独立的参照实现”是抓这类缺陷的唯一可靠办法。

4与开源实现的交叉对照

4.1 调研结果

按”与本题的可对照程度”排序：

项目	类型	与本题的关系
JammersSimulator-Tool github.com/Jammers-Simulator-Lab	MIT, Python	直接针对 2026 国赛 B 题。官方 Windows 模拟器的命令行/MCP 驱动，并附一个离线 mock 模拟器 (mock_simulator.py)。README 明确声明”仅含驱动，不含任何解法”，并以双盲评审为由做了脱敏。它是唯一能拿来逐项对照协议与计时的公开实现
dstl/Stone-Soup UK 国防科技实验室	BSD 类, Python	成熟的跟踪/估计框架，含纯方位跟踪示例 (RadarBearing 传感器 + Cartesian2DToBearing 观测模型 + 扩展卡尔曼滤波)。对照价值在方法论：它代表”把方位测量喂给贝叶斯滤波器递归估计目标状态”的路线；本题我们走的是确定性集合路线（角楔求交得到定位区域，最小包围圆给不确定度），不需要先验与过程模型，且能给出可证明的终止判据
tiiuae/gnn-jamming-source-localization 阿联酋 TH	研究代码	用图神经网络做干扰源定位，属同一应用域（RF 干扰源）但方法为数据驱动，需要仿真数据集训练；与本题的几何/在线决策路线互补
dressel/FEBOL.jl	Julia 研究代码	Filter Exploration for Bearing Only Localization，研究”为了定位该往哪里走”，和问题二的站位优化是同一类问题
El-Mohr/WSN-Localaization-Simulator	Python	无线传感器网络定位仿真环境，可作测距/测角定位的可视化参照
daniyalshagirotff/acoustic-triangulation	Python	声学三角定位求解器，其三角交会求解与本文问题一的角楔求交是同一数学内核

诚实说明

除第一项外，其余项目我只核对了其公开说明与示例，没有逐行审阅源码，因此上面的定位是“可对照性评估”而非“代码级比对结论”。第一项因为直接对应本题协议，做了逐项常量与成本模型比对（见下）。

4.2 逐项常量对照：12/12 相等

量	含义	参考 mock	本实现
速度	移动	5.0 m/s	5.0 m/s
切换	频道切换	1.0 s（同频道 0）	同
检测	一次测向	5.0 s	同
清除命中	光学 3 + 激光 2	5.0 s	同
清除未命中	仅光学	3.0 s	同
near	过近阈值	5.0 m	同
清除半径	与角度无关	20.0 m	同
场地半径	目标域	1800 m	同
接收半径	每源	[1000, 1500] m	同
源个数	每局	[10, 16]	同
虚拟时长上限	—	360000 s	同
真实时长上限	—	1200 s	同

4.3 成本模型：300 步随机动作逐步相等

把参考 mock 的计费公式（移动 $d/5$ + 切换 $[0, 1]$ + 检测 5；清除移动 $d/5$ + 光学 3 + 激光 $[0, 2]$ ）独立实现为参照，然后让本模拟器执行 300 次随机 /measure 与约 90 次随机 /clear，逐步比较虚拟时间增量：

检查	结果
measure_cost_matches_reference	最大偏差 $< 10^{-9}$ s
clear_cost_matches_reference	最大偏差 $< 10^{-9}$ s

两个独立实现在同一串动作上给出逐步相同的虚拟时间——这是“时间记账忠实”最强的证据。

4.4 唯一分歧：示向度误差模型

	参考 mock	本实现
实现	<code>err = rng.uniform(-1,1)</code>	<code>crc32(位置, 频道)</code> 映射到 <code>[-1, 1]</code>
每次调用	重新随机	完全相同
附件 1 要求		”同一地点重复测量结果相同”
是否符合题设	否	是

这一分歧很重要，因为它会影响策略选择：如果每次重测都重新随机，那么”在同一点多测几次取平均”就能降低误差，站位与信息获取的模型都要改写。该 mock 的定位是”离线开发与 CI 用的简化桩”（其 README 自述），我们没有理由跟随它偏离附件 1。

本实现的误差模型为何刻意”无空间结构”

误差是 (位置, 频道) 的纯函数, 按 0.01 m 取整后哈希。这意味着相邻地点的误差完全独立——现实中测向误差常有空间相关性（同一建筑反射造成同向偏差）。我们刻意不引入这种结构，以免结论建立在”利用了现实中并不存在的平滑性”之上。这是一次保守选择。

5 一个把两轮调参全部作废的陷阱

5.1 现象

第一轮坐标下降给出的结果好得可疑：

```
1 [q3] baseline ratio=1.0000 time=324.63
2 [q3] pass0 ring_radius = 1250.0 -> ratio=1.0000 time=181.06
3 [q3] FINAL ratio=1.0000 time=181.06
```

代码 2 第一轮寻优输出（有问题）

一个 1280 → 1250 的站位半径改动，把平均时间砍掉 44%？在独立算例池上复算：

```
1 Q3 baseline (rho=1280) ratio=1.0000 time= 331.8 travel=14392
2 Q3 tuned (rho=1250) ratio=1.0000 time= 326.2 travel=14097
```

代码 3 独立池（种子 31000-31039）复算

只有 1.7%，不是 44%。

5.2 根因与最小复现

`sim.Arena(sources)` 保存的是传入的 `Source` 对象本身（浅拷贝），运行中被清除的源会把 `cleared=True` 写在共享对象上。而寻优脚本为了”每个候选都在同一批算例上评估”，把算例池只生成了一次：

```
1 cases = pool(tag, n_cases)           # 只生成一次
2 for name, values in grid.items():
3     sc = evaluate(cand, cases)        # 每次都在同一批 Source 对象上跑
4                                     # -> 第 2 次起，源已经被清空了
```

代码 4 缺陷写法

最小复现（5 个算例，连跑两次）：

```
1 reused pool eval0: ratio=1.000 time=334.3
2 reused pool eval1: ratio=1.000 time=188.2 <-- 假的"提升"
3 regenerated eval0: ratio=1.000 time=334.3
4 regenerated eval1: ratio=1.000 time=334.3 <-- 真实
```

代码 5 复用池 vs 重新生成

第二轮寻优同样中招。这不是新问题——论文自检报告里记录的故障 1 就是它，当时修的是”调参脚本”，但新写的寻优脚本又犯了一次。说明光靠”上次修过”不够，必须让脚本自己拒绝这种状态。

5.3 修法与守卫

1. 每次评估都从种子重新生成算例（生成成本几毫秒，相对单局 1–3 s 可忽略）；
2. 加可复现性守卫：启动时对同一配置连算两次，若分数不一致则直接退出（返回码 2）并打印诊断：

```
1 [guard] ok: repeated evaluation is reproducible (1.0000, 1490.6 s)
```

代码 6 守卫的输出

5.4 修正后的真实收益

在从未参与过调参的独立算例池上（问题三种子 12000 起，问题四种子 62000 起）比较”优化前配置”与”验证后配置”——即同时计入本轮的三项算法改进（增量认证、定向场景下的无谓搜索站位抑制、Or-opt）：

	完成率	最差完成率	平均时间/s	平均行程/m
问题三优化前	1.0000	1.0000	见 §6 表	—
问题四优化前	0.9974	0.9375	1395.6	78202
问题四优化后	1.0000	1.0000	1114.0	见 §6

结论：问题三的性能是结构性的——8 个参数在 313~348 s 的窄带内几乎无差异，说明瓶颈是”覆盖 + 认证”的固有行程，不是参数没调好；问题四的收益是真实的，而且必须分两步看。

第一步：修正的代价。修正认证扫描后，”提前宣布完成”不再发生，机器人必须真正走完搜索，因此时间上升：

	完成率	最差完成率	平均时间/s	平均行程/m
修正前（错误认证）	0.9574	0.8333	661.0	28476
修正后	0.9935	0.9091	1476.1	73064

第二步：在正确的前提下重新优化。把搜索成本按任务类别拆开后发现：**89% 的时间与行程都花在”搜索站位”上**（65 次搜索任务对 12 次清除任务），而其中相当一部分站位是”域内永远无法完成认证”的边界候选点。

据此做了两项改动并逐项验收：

改动	完成率	平均时间/s	平均行程/m
（基线：修正后）	0.9944	1176.5	58944
抑制无法完成认证的搜索站位	0.9944	955.0	47923
再调参（格点间距 1100 m 等）	0.9935	945	47000

关键否证：一度实现的”边界外定向补测”（rim patrol）看似必要（定理 2 说边界点在域内不可认证），但在演练池 30 例上逐项对照后发现：开启与关闭的**完成率完全相同（0.9944，最差也相同 0.9000）**，而关闭后时间与行程各降 **18.8%**。逐个检查失败算例也显示，漏掉的源半径多在 1158~1673 m（并非贴边），补测机制对它们毫无帮助。因此该机制**默认关闭**，只在需要时作为可选项保留。

最终验收（独立算例池，从未参与调参）

问题四：平均时间 1476.1 s → **1018.1 s**（-31.0%），行程 -32.8%，完成率 0.9935 → 0.9893（相当）。问题三：平均时间 333.4 s → **323.0 s**（-3.1%），行程 -9.3%，完成率恒为 1.0000。正式测试（HTTP）：问题四由”11/11、11/12、11/11” 变为 **11/11、12/12、11/11**，此前漏掉的贴边朝外源被成功清除；问题三保持 15/15、12/12、10/10。

唯一未能消除的失败模式：贴边且定向方向朝外的源，在域内**数学上不可认证**（定理 2）。此时机器人无法证明该频道不存在，只能继续搜索直至预算耗尽。是否为此付出域外补测的代价（时间约 ×2.4）是一个策略选择，本文默认不付出，并在正文中如实说明。

6 本轮算法改动清单

改动	动机	验证方式与效果
增量认证扫描	原实现是 $O(\text{网格} \times \text{频道} \times \text{读数})$ ，单局 CPU 41 s	第 G 组逐次与暴力重算比对；CPU 41 s → 2.8 s，且修正了 <code>uncov</code> 被截断的缺陷
抑制无谓搜索站位	定向场景下，边界候选点附近所有域内方向都指向圆心，这类站位永远无法完成认证却照样消耗行程	10 例 A/B：时间 -8.5%、行程 -8.7%、完成率不变（1.0000）
边界外定向补测	边界候选点在域内 数学上不可认证 （定理 2）	第 G 组检查补测点确在域外、数量受限、内部候选不触发；使完成率在困难算例上达到 1.0000
Or-opt 局部搜索	最近邻 + 2-opt 之后仍有可改进的短段	初期版本在双重循环里全量重算路径长度，导致单局 CPU 数十秒；改为只优化序列前 12 个任务（滚动时域每次都会重规划，远期任务无意义）

7 如何复现全部验证

```
1 cd code
2 python verify_simulator.py      # 64 项检查 + 报告 data/simulator_verification.json
3 python optimize_q34.py q3 24 2 # 坐标下降（自带可复现性守卫）
4 python optimize_q34.py q4 24 2
5 python final_validate.py 40     # 独立池验收，写 data/final_validation.json
```

仍未做到的事

没有官方模拟器上的成绩。本报告能证明的最强命题是：“本模拟器在 64 项可检查的协议、计时与物理规定上全部通过，且其成本模型与一份独立开源实现逐步一致”——这**不等于**“在官方模拟器上会得到同样的分数”。若要最终确认，需要把 `HTTPClient` 指向官方运行时（默认端口相同），策略代码无需任何改动。